

KisanSetu — Production Build Mission Plan

Prototype Status:  COMPLETE (SIH 2026 Idea Phase Demo)

Mission: Transform the working prototype into a production-grade, deployable platform.

Team Size: 6 Members

Approach: Layered build — each layer completes before the next begins, but members overlap during transitions.

Current Prototype Stack (What We Have)

Component	Tech	Status
Frontend	React 18 + Vite + TailwindCSS	 15 pages, mock+real hybrid
Backend API	Express.js + SQLite	 11 controllers, 7 services
AI Microservice	Python Flask + sklearn	 Demand forecast + 2-Opt TSP
Voice IVR	Twilio + Sarvam AI TTS/STT	 Tamil phone call flow
Maps & Routing	OSRM + Nominatim	 Geocoding + road geometry
Auth	JWT + bcrypt	 Basic RBAC (farmer/consumer/driver/admin)
Database	SQLite (single file)	 Not production-ready
Payments	None	 Not built
Deployment	localhost only	 Not deployed
Mobile App	None (web-only)	 Not built
Testing	Manual + 4 PowerShell scripts	 No automated CI

Team Roster & Role Assignment

#	Role	Primary Ownership	Key Skills Needed
M1	Backend Architect	Database, API, Auth, Payment Gateway	Node.js, PostgreSQL, Razorpay, Redis
M2	Frontend Lead	Web UI, PWA, Responsive Design	React, TailwindCSS, Recharts, Leaflet
M3	AI/ML Engineer	ML Models, NLP, Voice AI Pipeline	Python, scikit-learn, PyTorch, Sarvam AI
M4	Telephony & Integration Engineer	IVR, SMS, Twilio, WhatsApp Bot	Twilio, WebSockets, Sarvam STT/TTS
M5	Mobile & DevOps Engineer	React Native App, CI/CD, Cloud Deploy	React Native, Docker, Render/Vercel, GitHub Actions
M6	QA & Data Engineer	Testing, Data Pipeline, APMC Feed, Analytics	Jest, Pytest, Selenium, SQL, Grafana

Build Layers (Sequential Execution)

Each layer has a **duration**, **owner(s)**, **deliverables**, and **acceptance criteria**.

LAYER 1: Foundation & Database Migration

Duration: Week 1–2 (14 days)

Lead: M1 (Backend Architect)

Support: M6 (Data Engineer)

[!IMPORTANT] Everything else depends on this layer. No frontend or AI work can ship to production without a real database and proper auth.

Deliverables

M1 — Backend Architect

- ☐ **PostgreSQL Migration**
 - Replace SQLite with PostgreSQL 16
 - Design normalized schema: `users` , `products` , `orders` , `cart_items` , `deliveries` , `payments` , `ivr_logs` , `market_prices` , `reviews`
 - Write migration scripts (use [Knex.js](#) or [Prisma](#))
 - Add proper indexes, foreign keys, cascading deletes
 - Connection pooling with `pg-pool` (max 20 connections)
- ☐ **Auth & Security Hardening**
 - Phone OTP login (Twilio Verify API) — no more password-only
 - Refresh token rotation (access token: 15 min, refresh: 7 days)
 - Rate limiting: `express-rate-limit` (100 req/min per IP, 10 req/min for auth)
 - Input sanitization: `express-validator` on every route
 - Helmet.js security headers
 - CORS whitelist (only production domains)
- ☐ **Environment & Config**
 - `.env.development` , `.env.staging` , `.env.production`
 - Secrets in environment variables (never in code)
 - Structured logging with [Pino](#)

M6 — Data Engineer

- ☐ **Seed Data Pipeline**
 - Migrate all existing SQLite seed data to PostgreSQL
 - Write seed scripts for 50+ demo products, 10 farmers, 5 consumers, 3 drivers
 - Import real APMC Agmarknet price data (CSV → PostgreSQL `market_prices` table)
 - Create `data/migrations/` folder with versioned SQL files

Acceptance Criteria

- `npm run migrate` creates all tables in PostgreSQL
- `npm run seed` populates demo data

- All existing API routes pass with PostgreSQL backend
- Auth flow works with phone OTP

LAYER 2: Core API Rebuild & Payment Integration

Duration: Week 2–4 (14 days)

Lead: M1 (Backend Architect)

Support: M4 (Integration Engineer)

Deliverables

M1 — Backend Architect

- ☐ **Payment Gateway (Razorpay TEST MODE — ₹0 cost)**
 - Razorpay Test Mode: Full API, no real money moves, unlimited test transactions
 - `POST /api/payments/create-order` → Razorpay order creation (test keys)
 - `POST /api/payments/verify` → Signature verification
 - `POST /api/payments/webhook` → Razorpay webhook handler
 - Payment states: `pending` → `captured` → `settled` (to farmer) / `refunded`
 - Escrow logic: Hold payment until delivery confirmed, then release 98% to farmer bank
 - UPI Autopay for recurring bulk orders (restaurants/hostels)
 - Switch to production keys only when real money needed (post-SIH)
- ☐ **Order Lifecycle Engine**
 - States: `placed` → `confirmed` → `farmer_packed` → `driver_picked` → `in_transit` → `delivered` → `settled`
 - Real-time status updates via Server-Sent Events (SSE) or WebSocket
 - Auto-cancel if farmer doesn't confirm within 2 hours
 - Dispute resolution: `disputed` → `admin_review` → `resolved`
- ☐ **Farmer Payout Service**
 - Razorpay Route (split payment) or manual payout via `POST /api/payouts`
 - Farmer bank account verification (IFSC + account number)
 - Payout ledger table: tracks every ₹ in and out
 - Settlement cycle: T+1 after delivery confirmation

M4 — Integration Engineer

- ☐ **SMS & Notification Service (Free Tier)**
 - Continue on Twilio Trial (\$15 credit — sufficient for demo + early users)
 - Use Twilio `<Say>` Polly.Aditi as primary voice (unlimited, no download cost)
 - Templates: Order placed, Order confirmed, Out for delivery, Delivered, Payment received
 - SMS in Tamil, Hindi, English (template-based, not dynamic)
 - Push notification infrastructure (Firebase Cloud Messaging — free, unlimited)

Acceptance Criteria

- End-to-end payment: Consumer pays → Escrow holds → Delivery confirmed → Farmer receives 98%
- Order status updates in real-time on buyer dashboard

- SMS confirmations sent at every lifecycle stage
- Payment webhook handles duplicate/retry scenarios

LAYER 3: AI/ML Production Pipeline

Duration: Week 3–5 (14 days)

Lead: M3 (AI/ML Engineer)

Support: M6 (Data Engineer)

Deliverables

M3 — AI/ML Engineer

- ☐ **Demand Forecasting Model (Production)**
 - Replace dummy RandomForest with XGBoost/LightGBM trained on real APMC historical data
 - Features: commodity, season, region, festival calendar, MSP announcements, rainfall index
 - Train on 3+ years of Agmarknet data (2023–2026)
 - Serve via FastAPI with model versioning (`/v1/predict-demand`)
 - Weekly auto-retrain cron job
- ☐ **Smart Matching Engine**
 - Proximity-weighted scoring: distance (40%), price competitiveness (25%), freshness (20%), farmer rating (15%)
 - PostgreSQL full-text search for product discovery
 - Bulk buyer preference learning (what does this restaurant order every week?)
- ☐ **Crop Doctor AI**
 - Image classification model for leaf disease detection (ResNet50 transfer learning)
 - Dataset: PlantVillage (54K images, 38 classes)
 - API: `POST /api/ai/diagnose` with image upload → disease name + treatment
 - Voice-based symptom description (Sarvam STT → LLM → treatment advice in Tamil)
- ☐ **Route Optimization (Production)**
 - Replace mock OSRM with self-hosted or Mapbox Directions API
 - Multi-stop optimization: 2-Opt TSP with time windows (perishability constraints)
 - Vehicle capacity constraints (Tata Ace: 750 kg, Eicher: 4000 kg)
 - Real-time traffic-aware ETA

M6 — Data Engineer

- ☐ **Data Pipeline**
 - Daily APMC price scraper (Agmarknet → PostgreSQL)
 - Weather data feed (IMD API → rainfall, temperature)
 - Historical training data preparation scripts
 - Model performance monitoring dashboard (accuracy, drift detection)

Acceptance Criteria

- Demand forecast within $\pm 15\%$ accuracy on test set

- Crop Doctor identifies top-5 diseases with >80% accuracy
- Route optimization handles 20+ stops in <2 seconds
- Smart match ranks products meaningfully (not random)

LAYER 4: Frontend Production Build & PWA

Duration: Week 4–6 (14 days)

Lead: M2 (Frontend Lead)

Support: M3 (AI/ML — for AI Insights page integration)

Deliverables

M2 — Frontend Lead

- ☐ **UI/UX Overhaul**
 - Design system: Component library with consistent spacing, colors, typography
 - Dark mode support
 - Full responsive design (mobile-first): 320px → 1920px
 - Accessibility: ARIA labels, keyboard navigation, screen reader support
 - Loading skeletons on every data-fetching page
 - Error boundaries with friendly fallback UI
- ☐ **PWA (Progressive Web App)**
 - Service worker for offline caching (product catalog, last-viewed prices)
 - Web App Manifest: installable on Android home screen
 - Push notifications via Firebase
 - Offline-first cart (sync when back online)
- ☐ **Real-Time Features**
 - Live order tracking map (WebSocket-driven truck position updates)
 - Real-time price ticker on marketplace (SSE from backend)
 - Notification bell with unread count
- ☐ **Page Rewrites (Production Quality)**
 - `Marketplace.jsx` — Infinite scroll, advanced filters, sort persistence in URL
 - `ProductPage.jsx` — Image gallery, reviews section, "Similar Products" carousel
 - `BuyerDashboard.jsx` — Real payment history, downloadable invoices (PDF)
 - `FarmerDashboard.jsx` — Revenue charts (daily/weekly/monthly), export to CSV
 - `LogisticsDashboard.jsx` — Real GPS tracking, delivery proof photo upload
 - `DialphoneGateway.jsx` — Live call status websocket, call recording playback

Acceptance Criteria

- Lighthouse score: Performance >80, Accessibility >90, PWA >90
- All pages functional on Chrome Android, Safari iOS, Desktop Chrome/Firefox
- Offline mode: Can browse cached products and manage cart without internet
- No console errors, no broken images, no layout shifts

LAYER 5: Mobile App & Telephony Production

Duration: Week 5–7 (14 days)

Lead: M5 (Mobile & DevOps)

Co-Lead: M4 (Telephony & Integration)

Deliverables

M5 — Mobile & DevOps Engineer

- ☐ **React Native App (Android-first)**
 - Screens: Login (OTP), Home Feed, Marketplace, Product Detail, Cart, Orders, Profile
 - Farmer mode: List produce with camera (photo → crop detection AI)
 - Driver mode: Accept delivery, navigate (Google Maps intent), scan QR at pickup/delivery
 - Push notifications (FCM)
 - Offline SQLite cache for poor-connectivity rural areas
 - APK build + Play Store internal testing track
- ☐ **Cloud Infrastructure (100% FREE TIER)**
 - PostgreSQL: [Neon](#) free tier (0.5 GB, auto-suspend)
 - Backend API: [Render](#) free web service (750 hrs/mo, auto-sleep)
 - AI Service: [Hugging Face Spaces](#) free CPU (2 vCPU, 16 GB)
 - Redis: [Upstash](#) free (10K commands/day, session store + rate limiting)
 - File Storage: [Supabase Storage](#) free (1 GB — product images, delivery proofs)
 - Frontend: [Vercel](#) free (unlimited static, 100 GB bandwidth, auto SSL)
 - CDN + DNS: [Cloudflare](#) free (unlimited bandwidth, DDoS protection)
 - Domain: [kisansetu.onrender.com](#) (free) or buy [.in](#) domain later (~₹500/year if needed)
 - Keep-alive: UptimeRobot pings Render every 14 min to prevent cold starts during demo
- ☐ **CI/CD Pipeline**
 - GitHub Actions: lint → test → build → deploy (staging on PR, production on main)
 - Docker Compose for local development
 - Dockerfiles for backend, ai-service, frontend (nginx)
 - Environment-specific builds (dev/staging/prod)

M4 — Telephony & Integration Engineer

- ☐ **IVR Production Deployment**
 - Twilio Trial account (free \$15 credit for demo calls)
 - Webhook hosted on Render backend (not localhost tunnel)
 - Call recording storage in Supabase Storage (1 GB free)
 - IVR analytics dashboard: calls/day, avg duration, drop-off points
 - Multi-language IVR: Tamil, Hindi, Telugu, Kannada (Sarvam AI voices)
- ☐ **WhatsApp Business Bot**
 - Twilio WhatsApp Sandbox (free for demo/testing)

- Commands: `SELL <crop> <qty> <price>` , `ORDERS` , `PRICE <crop>`
- Rich message templates with buttons (Confirm Order, Track Delivery)
- Image-based crop listing (farmer sends photo → AI categorizes)

Acceptance Criteria

- Android APK installable and functional on budget phones (2GB RAM, Android 10+)
- Backend auto-scales from 0 to 50 concurrent users
- CI/CD deploys to staging in <5 minutes
- WhatsApp bot responds within 3 seconds
- IVR handles 10 concurrent calls without dropping

LAYER 6: Testing, Analytics & Launch Readiness

Duration: Week 7–8 (14 days)

Lead: M6 (QA & Data Engineer)

Support: ALL MEMBERS

Deliverables

M6 — QA & Data Engineer

- ☐ **Automated Test Suite**
 - Backend: Jest + Supertest — 80%+ code coverage on controllers
 - Frontend: React Testing Library + Vitest — critical user flows
 - AI Service: Pytest — model accuracy assertions
 - E2E: Playwright or Cypress — 10 critical user journeys:
 1. Consumer registers → browses → adds to cart → pays → tracks delivery
 2. Farmer registers → lists produce via web → receives order → confirms
 3. Farmer calls IVR → lists produce via voice → product appears in marketplace
 4. Driver accepts delivery → navigates → delivers → uploads proof
 5. Admin views dashboard → sees all orders/users/revenue
 6. Consumer searches → filters by organic/category → sorts by price
 7. Farmer checks earnings → sees payout history
 8. Bulk buyer places large order → gets tiered discount
 9. WhatsApp bot: SELL command → listing created
 10. Market prices page shows real APMC data
- ☐ **Performance & Load Testing**
 - Artillery or k6 load test: 100 concurrent users, <500ms p95 response time
 - Database query optimization: EXPLAIN ANALYZE on slow queries
 - Image optimization: WebP conversion, lazy loading audit
- ☐ **Analytics & Monitoring**
 - Application monitoring: Sentry (error tracking)
 - Uptime monitoring: BetterUptime or UptimeRobot
 - Business analytics dashboard (Grafana + PostgreSQL):

- Daily GMV, active farmers, active consumers
 - Average order value, repeat purchase rate
 - IVR call volume, conversion rate (call → listing)
 - Delivery success rate, average delivery time
- Google Analytics 4 on frontend

ALL MEMBERS

- ☐ **Security Audit**
 - OWASP Top 10 checklist
 - SQL injection testing (parameterized queries verified)
 - XSS prevention (React auto-escapes, CSP headers)
 - API authentication on every protected route
 - File upload validation (type, size, malware scan)
- ☐ **Documentation**
 - API documentation: Swagger/OpenAPI spec
 - Architecture diagram (Mermaid in README)
 - Deployment runbook: How to deploy from scratch
 - User guide: For farmers (Tamil/Hindi), for consumers, for drivers

Acceptance Criteria

- All 10 E2E tests pass on staging
- Zero critical/high Sentry errors in 48-hour soak test
- Load test: 100 users, <500ms p95, 0% error rate
- Grafana dashboard shows live metrics
- Documentation reviewed by all team members

Master Timeline (8-Week Sprint)

Week 1-2		LAYER 1: Foundation (M1, M6)
Week 2-4		LAYER 2: API & Payments (M1, M4)
Week 3-5		LAYER 3: AI/ML Pipeline (M3, M6)
Week 4-6		LAYER 4: Frontend PWA (M2, M3)
Week 5-7		LAYER 5: Mobile & Deploy (M5, M4)
Week 7-8		LAYER 6: QA & Launch (M6, ALL)

[!NOTE] Layers overlap by 1 week — the lead starts the next layer while the support finishes the previous one. This keeps all 6 members busy at all times.

Member Workload Across Weeks

Week	M1 (Backend)	M2 (Frontend)	M3 (AI/ML)	M4 (Telephony)	M5 (Mobile/DevOps)	M6 (QA/Data)
1	L1 Lead	Study prototype	Study prototype	Study prototype	Setup Docker	L1 Support
2	L1 → L2	Study UX patterns	Collect training data	Study Twilio prod	Setup CI scaffold	L1 finish
3	L2 Lead	Start component lib	L3 Lead	L2 Support	Docker + infra	L3 Support
4	L2 finish	L4 Lead	L3 → L4 support	L2 finish → study	Infra + RN scaffold	L3 data pipeline
5	Code review	L4 Lead	L4 AI pages	L5 Co-Lead	L5 Lead	Data pipeline done
6	Payment edge cases	L4 finish	Model tuning	L5 IVR prod	L5 Mobile	Start test suite
7	Bug fixes	Bug fixes	Bug fixes	L5 WhatsApp	L5 Deploy	L6 Lead
8	Security audit	Lighthouse audit	Model validation	IVR load test	Prod deploy	L6 Launch



Production Infrastructure — ₹0 / ZERO COST (All Free Tiers)

[!IMPORTANT] Every single service below is **completely free**. No credit card required for most. Enough capacity for SIH demo + initial 100–500 real users.

Service	Free Provider	Free Tier Limit	Replaces
PostgreSQL Database	Neon	0.5 GB storage, 190 compute-hours/mo	AWS RDS
Backend API Hosting	Render	750 hrs/mo (sleeps after 15 min idle)	AWS ECS
Frontend Hosting	Vercel	Unlimited static, 100 GB bandwidth	CloudFront
AI Service Hosting	Hugging Face Spaces	2 vCPU, 16 GB RAM (free CPU)	GCP Cloud Run
Redis / KV Store	Upstash	10K commands/day, 256 MB	ElastiCache
File Storage (images)	Supabase Storage	1 GB storage, 2 GB bandwidth	S3
CDN + DNS + SSL	Cloudflare	Unlimited bandwidth, free SSL	CloudFront
Voice Calls (IVR)	Twilio Trial	\$15 free credit (~50 calls)	Twilio Paid
SMS	Twilio Trial	Included in \$15 credit (~100 SMS)	MSG91
Sarvam AI (TTS/STT)	Sarvam Free Tier	1000 API calls/mo	Sarvam Paid
Voice (fallback)	Twilio <Say> Polly.Aditi	Unlimited (no audio download)	Sarvam TTS
CI/CD	GitHub Actions	2000 min/mo (public repo)	Jenkins
Error Tracking	Sentry	5K events/mo	Paid Sentry
Uptime Monitoring	UptimeRobot	50 monitors, 5-min checks	BetterUptime
Analytics	Google Analytics 4	Unlimited	Mixpanel
Push Notifications	Firebase Cloud Messaging	Unlimited	OneSignal
Maps & Geocoding	Leaflet + Nominatim	Unlimited (open source)	Google Maps
Domain	Freenom / Render subdomain	kisansetu.onrender.com	Paid domain
WhatsApp Bot	Twilio WhatsApp Sandbox	Free sandbox (demo only)	WhatsApp Business
TOTAL			₹0/month

[!TIP] **Render cold start:** Free Render backend sleeps after 15 min of inactivity. First request takes ~30s to wake. Acceptable for SIH demo. Use Upstash Redis cron ping to keep it alive during demo hours if needed.

[!NOTE] **When to upgrade (post-SIH, if you get funding):** Neon → Supabase Pro (₹2K/mo), Render → Render Starter (₹600/mo), Hugging Face → GPU Space (₹1.5K/mo). Total ~₹4K/mo for 1000+ users.

Definition of Done (Production Launch Checklist)

- ☐ All 6 layers completed and tested
- ☐ Staging on Render mirrors production setup
- ☐ 10 E2E tests pass on staging
- ☐ Load test: 100 users, <500ms, 0% errors

- ☐ Security audit: OWASP Top 10 passed
- ☐ Payment flow tested end-to-end in Razorpay Test Mode
- ☐ IVR call works via Twilio Trial number
- ☐ Android APK tested on 3+ devices
- ☐ Monitoring & alerting live (Sentry free + UptimeRobot)
- ☐ Documentation complete (API, deploy, user guide)
- ☐ Live URL: kisansetu.onrender.com (free subdomain)
- ☐ Team demo: Full end-to-end walkthrough recorded

Total infrastructure cost: ₹0/month. Every tool in this plan is free-tier. Zero money needed to build, deploy, and demo.